

Assignment 06: Multimodal Search System for E-Commerce

1. Introduction

An e-commerce search system should accept different forms of customer input while maintaining a consistent retrieval process. A customer may type a keyword, speak a product description, or provide a product image. This project designs and implements a small prototype that connects requirements analysis, three-layer architecture, UML modeling, and executable Python code, as specified in Assignment 06.

The central design principle is:

Different Inputs -> Common Query Representation -> Retrieval -> Ranking -> Results

The required implementation supports text, simulated voice, and image-similarity search. It uses a local dataset of ten products, ten sample images, and three orders. The purpose is to demonstrate a coherent software design rather than a production shopping platform. Optional extensions include text-and-image fusion, filtering, stock-aware scoring, and order lookup.

2. Problem Description

The Customer searches a catalog containing shoes, bags, headphones, and watches. Product results include relevance scores and prices. Order queries return an order identifier, customer identifier, date, status, and total.

Table 1. Supported input modalities

Mode	Input	Processing	Output
Text	Keywords or a natural-language sentence	Normalize text, retrieve matching products, rank candidates	Ranked products and scores
Voice	A supplied transcript representing speech-to-text output	Validate transcript, construct a voice query, reuse text retrieval	Ranked products and scores
Image	A local product-image file	Encode image, compare vectors using cosine similarity, rank candidates	Visually similar products and scores
Multimodal extension	Text and an image	Combine text and image scores	Ranked products with both score components
Order extension	Order ID or latest-order request	Look up order data	Order information or an explicit no-result message

The console voice mode explicitly simulates speech-to-text. The website can request microphone input through a compatible browser’s Web Speech API and submits the resulting transcript. The image pipeline processes actual sample files using lightweight color and spatial features by default.

3. Requirements Analysis

3.1 Actor

Table 2. Actor description

Actor	Description	Main interactions
Customer	A user seeking product or order information	Submit text, voice, or image queries; view products; search and view orders

3.2 Functional Requirements

Functional requirements describe observable operations of the system. The first seven requirements cover the core prototype; the remaining requirements cover the implemented extensions.

Table 3. Functional requirements

ID	Requirement
FR01	The system shall search products using keywords or natural-language text.
FR02	The system shall accept a simulated speech-to-text transcript for voice search.
FR03	The system shall search for products similar to an input product image.
FR04	The system shall convert supported inputs into a common query representation.
FR05	The system shall retrieve candidate products before ranking them.
FR06	The system shall rank retrieved products and return the highest-scoring results first.
FR07	The system shall display returned product information and ranking scores.
FR08	The system shall search orders by order ID and display the corresponding information.
FR09	The system shall find the latest order for a specified customer.
FR10	The system shall support maximum-price, color, and category filters.
FR11	The system shall combine text and image relevance in a multimodal query.

3.3 Non-Functional Requirements

Non-functional requirements describe quality expectations. These expectations are proposed for the prototype; they are not additional thresholds specified by the assignment handout.

Table 4. Non-functional requirements

ID	Requirement
NFR01	The prototype should return results within an acceptable time for a small local dataset.
NFR02	The interface should present inputs, processing descriptions, returned products, and scores in a readable and consistent format.
NFR03	The implementation should separate presentation, application, and data responsibilities to support maintenance.
NFR04	Speech and image processing should be replaceable without changing the presentation interface.
NFR05	Invalid input should produce a clear error instead of a misleading search result.
NFR06	Repeated searches with unchanged inputs and data should produce deterministic results.
NFR07	Search operations should be independently verifiable through automated tests.

NFR01 is a qualitative target; a hardware-specific latency benchmark has not been claimed. NFR03 and NFR04 are assessed by reviewing dependencies and service boundaries. NFR06 and NFR07 are assessed through the supplied automated tests.

3.4 Technical Constraints

Table 5. Implementation constraints and design choices

Item	Specification	Basis
Language	Python	Required by the assignment
Runtime	Python 3.10 or later	Implementation choice
Runtime packages	NumPy and Pillow	Implementation choice
UML tool	Visual Paradigm	Required by the assignment
Voice component	Simulated transcript	Permitted by the assignment
Product data	At least ten products	Required by the assignment
Storage	Local JSON and image files; in-memory vector index	Prototype design choice
Interface	Console commands and local Flask website	Web UI implements an optional advanced item

3.5 Input and Output Specification

Table 6. Query and result fields

Object	Fields	Purpose
QueryRepresentation	type, raw_input, normalized_text, image_path, image_vector, filters	Common envelope for text, voice, image, and multimodal input
Candidate	product, text_score, image_score	Retrieved product before final ranking
RankedResult	product, score, text_score, image_score, business_score, reason	Explainable ranked output
Order	order_id, customer_id, date, status, total	Order-search output

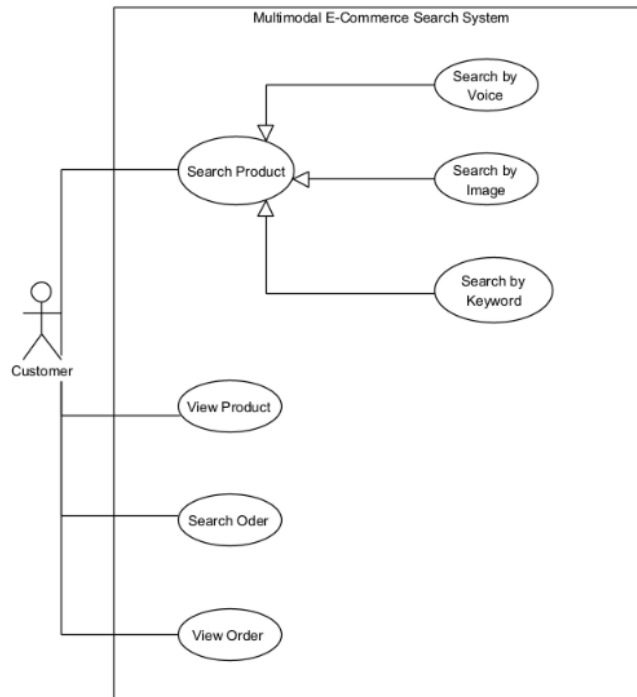
The common query object is a shared structure, not a claim that text and image use the same numerical feature space.

4. Use Case Model

4.1 Use Case Diagram

The figure specifies the UML drawing to reproduce in Visual Paradigm. Generalization arrows point from a specialized search use case to Search Product. Viewing details is modeled as a separately selectable action rather than a compulsory step in every search.

Figure 1. Product and order search use case model



4.2 Use Case Descriptions

Table 7. Required use cases

ID	Use case	Input/action	Successful outcome
UC01	Search Product	Customer selects a search modality	Ranked candidate products are available
UC02	Search by Keyword	Customer submits a text query	Matching products are retrieved and ranked
UC03	Search by Voice	Customer supplies a simulated transcript	Transcript is processed through text retrieval
UC04	Search by Image	Customer supplies an image path	Products are ranked by image-related relevance
UC05	Search Order	Customer submits an order identifier	Matching order is returned
UC06	View Product	Customer examines returned product information	Product name, price, and search scores are displayed
UC07	View Order	Customer examines an order result	Order identifier, date, status, and total are displayed

Table 8. Detailed voice-search scenario

Field	Description
Actor	Customer
Preconditions	The product dataset and image index have loaded successfully.
Trigger	Customer submits a non-empty simulated voice transcript.
Main flow	Validate transcript; normalize query; retrieve products; rank candidates; display results.
Alternative flow	No matching product: display a no-result message.
Exception flow	Empty transcript: raise a clear validation error.
Postcondition	Ranked product information is displayed; stored catalog data is unchanged.

5. Three-Layer Architecture

5.1 Layer Responsibilities

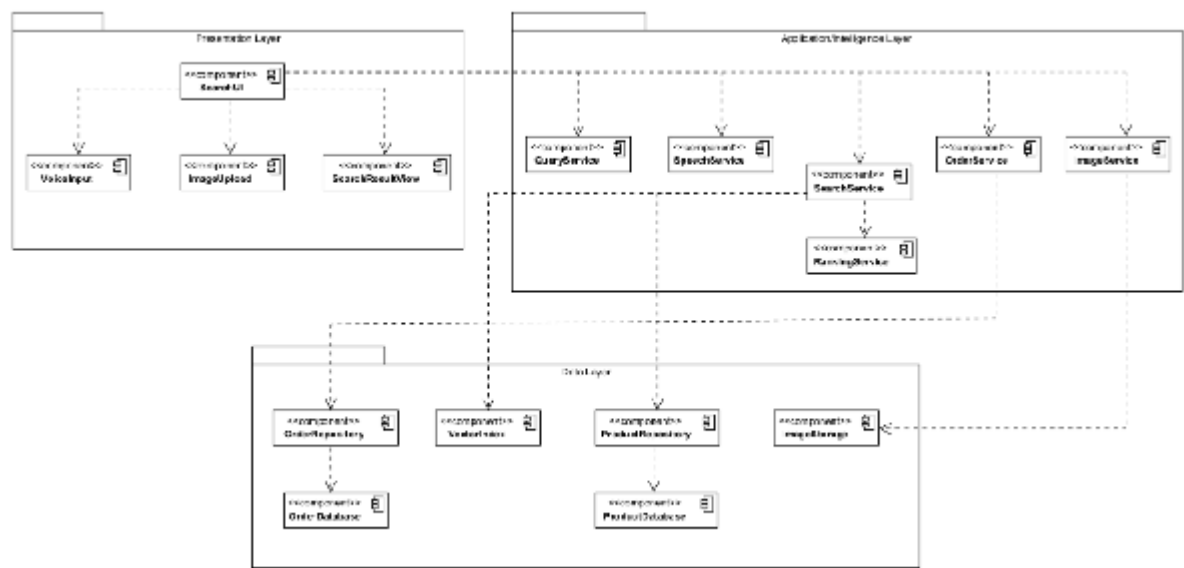
Table 9. Responsibilities and implementation mapping

Layer	Responsibility	Python location
Presentation	Accept command-line input and display search results	main.py and presentation/search_ui.py
Application / Intelligence	Construct queries, process modalities, retrieve candidates, rank results	application/
Data	Load product and order records; store and compare image vectors	data_access/ and dataset/

The composition root, `app_factory.py`, constructs and connects dependencies. Shared data objects reside in `domain/`. SearchUI does not read JSON files or access repositories.

5.2 Component/Package Diagram

Figure 2. Three-layer architecture



`VoiceInput`, `ImageUpload`, and `SearchResultView` are separate presentation classes used by `SearchUI`. `SearchUI` calls `ImageService` directly to encode an uploaded image, attaches the returned vector to the common query object, and submits the query to `SearchService`. `SearchService` accesses `VectorIndex` and `ProductRepository`, then calls `RankingService`. `OrderService` accesses `OrderRepository`. `ImageStorage` holds local PNGs; `ImageService` reads those files during encoding. The composition root builds catalog vectors before constructing `VectorIndex`. `ProductDatabase` and `OrderDatabase` are logical stores implemented with JSON, rather than database-server products.

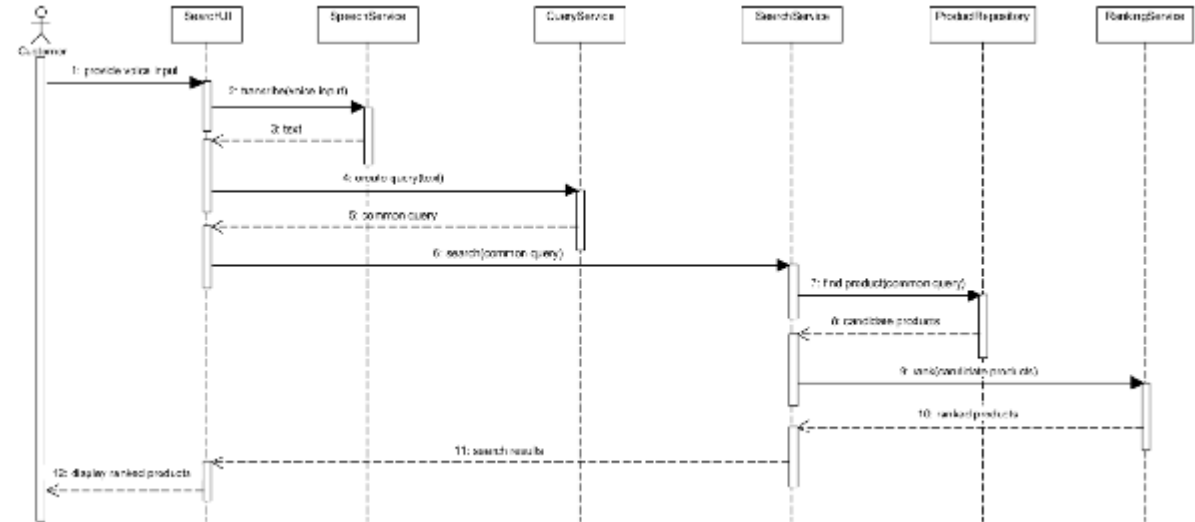
5.3 Component Responsibilities

Table 10. Component responsibilities

Component	Responsibility
SearchUI	Collect input and coordinate application-service calls
QueryService	Normalize text and construct a common query object
SpeechService	Validate and return the simulated transcript
ImageService	Encode image files into feature vectors
SearchService	Retrieve candidates and apply filters
RankingService	Calculate final scores and sort candidates
ProductRepository	Read and expose product records
OrderService	Interpret order requests and call OrderRepository
OrderRepository	Find orders by ID or latest customer date
VectorIndex	Store product vectors and calculate cosine similarities

6. UML Sequence Diagram

Figure 3. Voice-search interaction matching the implementation



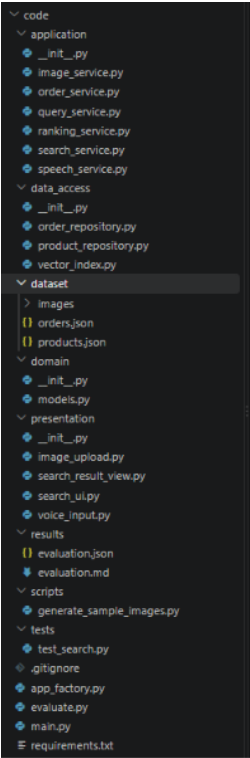
The seven participants explicitly required by the assignment are present. SearchUI collects the transcript through voice input, calls SpeechService and QueryService, and submits the common query to SearchService. SearchService retrieves products through ProductRepository and delegates ordering to RankingService. SearchUI display.

Table 11. Voice-search processing steps

Step	Component	Operation
1	SearchUI	Receive the voice-search request from Customer
3	SpeechService	Reject an empty transcript and trim whitespace
4	QueryService	Create a voice QueryRepresentation
5	SearchService	Retrieve matching candidates
6	ProductRepository	Return product data
7	RankingService	Order candidates and return them to SearchService

7. Python Implementation

7.1 Directory Structure



The handout's directory layout is recommended rather than mandatory. This project distinguishes data_access/ for repository code from dataset/ for stored records and images.

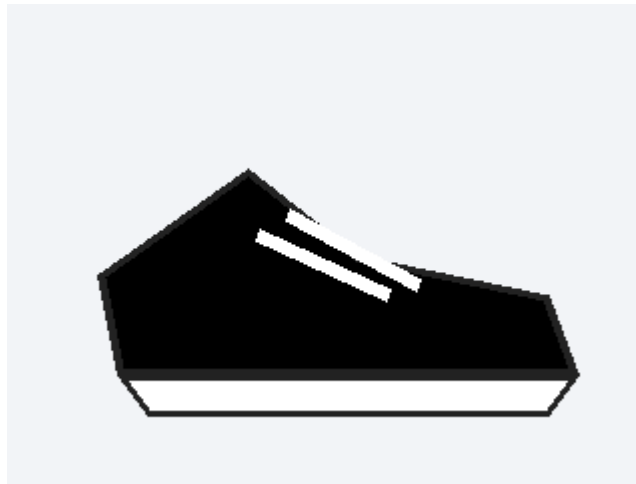
7.2 Product Repository and Dataset

ProductRepository loads records from products.json and rejects a catalog containing fewer than ten products. Each record includes id, name, category, color, price, stock, description, image, and popularity. Embeddings are derived from images at startup and stored in VectorIndex rather than duplicated in JSON.

Table 12. Product dataset

ID	Product	Category	Color	Price (USD)	Stock
1	Nike Black Running Shoes	shoes	black	120	10
2	Adidas White Running Shoes	shoes	white	100	15
3	Black Leather Bag	bag	black	80	20
4	Blue Sports Shoes	shoes	blue	75	12
5	Red Casual Shoes	shoes	red	65	8
6	Brown Travel Bag	bag	brown	95	7
7	Black Wireless Headphones	headphones	black	150	18
8	White Wireless Headphones	headphones	white	140	13
9	Black Smart Watch	watch	black	200	9
10	Green Sports Watch	watch	green	110	14

Figure 4. Example catalog image used in the image-search demonstration



These are generated illustrations supplied with the prototype, not photographs collected from a commercial catalog.

7.3 Query Service

QueryService produces a QueryRepresentation for every modality. Text processing lowercases input, normalizes characters, tokenizes words, and removes a small set of stop words. Known color and category terms become filters. A phrase such as under 110 dollars becomes a maximum-price constraint.

```
query = query_service.text_query("black running shoes")
voice_query = query_service.voice_query("find running shoes")
image_query = query_service.image_query(image_path)
```

7.4 Text Search

SearchService compares descriptive query words with the product name, category, color, and description. Color, category, brand, and maximum price are first extracted as constraints. A candidate must satisfy every extracted constraint and every remaining descriptive word. A filter-only query can return all eligible products, ordered by business score.

text_score = matched query words / total unique query words

This implementation follows the handout's simple keyword-matching approach while normalizing the score for fusion. Constraint words and numbers are excluded from the text denominator. Thus "nike shoes under 100 dollars" correctly returns no product in this catalog; Adidas shoes do not satisfy the Nike brand constraint.

7.5 Simulated Voice Search

```
class SpeechService:
def transcribe(self, audio_input: str) -> str:
if not audio_input.strip():
raise ValueError("The simulated voice transcript cannot be empty.")
return audio_input.strip()
```

In the console, the input is already a transcript. In the web interface, a compatible browser can capture speech using its Web Speech API and place the recognized transcript in the form. The Python server still receives text, so browser speech recognition quality is outside this evaluation.

7.6 Image-Similarity Search

The assignment permits artificial vectors for a first prototype. This implementation instead extracts simple features from real PNG files: 24 histogram values, comprising eight bins per RGB channel, and 27 spatial values, comprising mean RGB values for nine grid cells. The resulting 51-dimensional vector is normalized using its L2 norm.

cosine_similarity(a, b) = dot(a, b) / (norm(a) * norm(b))

A zero denominator produces a similarity of zero. VectorIndex compares the query vector with every catalog vector. ImageService performs feature extraction; VectorIndex stores and compares vectors without importing application services.

7.7 Result Ranking

Retrieval identifies candidates; RankingService calculates their final order. The core relevance score is text similarity for text/voice and cosine similarity for image search.

```
business_score = 0.7 * popularity + 0.3 * min(stock / 20, 1)
final_score = 0.98 * relevance + 0.02 * business_score
```

The business contribution is an optional extension. It has a small weight but can reorder sufficiently close relevance scores; it is not strictly limited to exact ties. Results are sorted by descending final score, then product ID for deterministic tie resolution. The default output contains up to five products.

7.8 Main Program and Execution

Table 13. Commands

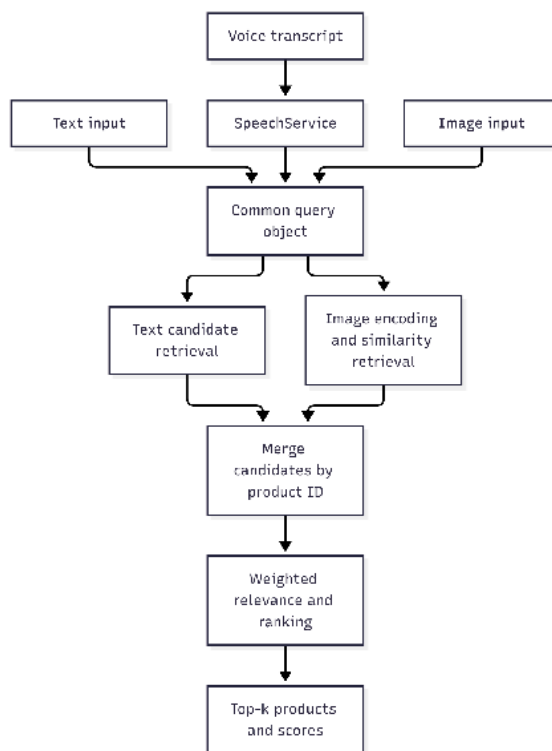
Purpose	Command, run inside code/
Install packages	python -m pip install -r requirements.txt
Complete demonstration	python main.py demo
Text search	python main.py text "black running shoes"
Simulated voice search	python main.py voice "find running shoes under 110 dollars"
Image search	python main.py image "dataset/images/nike_black_running_shoes.png"
Experimental evaluation	python evaluate.py

Purpose	Command, run inside code/
Automated tests	<code>python -m unittest discover -s tests -v</code>
Web interface	<code>python web.py</code> , then open <code>http://127.0.0.1:5000</code>

8. Multimodal Search Method

Supporting three separate modalities is required. Combining text and image in a single query is an implemented extension. The fusion method preserves separate feature spaces and combines scores after retrieval.

Figure 5. Multimodal processing flow



$\text{multimodal_relevance} = 0.5 * \text{text_score} + 0.5 * \text{image_score}$

Both weights sum to one, following the handout's fusion example. The combined relevance is then passed to the same final ranking formula used by individual modalities. Text-derived filters apply before final ranking.

9. Experimental Results

9.1 Demonstration of Required Queries

Table 14. Demonstration inputs, processing, and outputs

Mode	Input	Processing	Top returned product	Final score
Text	black running shoes	Normalize; filter; retrieve; rank	Nike Black Running Shoes	0.9963
Voice	find running shoes under 110 dollars	Simulated transcription; normalize; filter; retrieve; rank	Adidas White Running Shoes	0.9971
Image	nike_black_running_shoes.png	Encode; cosine retrieval; rank	Nike Black Running Shoes	0.9963
Multimodal	black shoes + Nike sample image	Text/image retrieval; fusion; rank	Nike Black Running Shoes	0.9963

The image query obtains a cosine similarity of 1.000 for its identical catalog image. The final score is slightly lower because it also includes business information.

The current console demonstration can be reproduced with `python main.py demo` inside `code/`. It prints each input, a processing summary, returned products, and ranking scores. The web evidence below shows the same search service through the browser interface.

9.2 Experimental Evaluation

A test is successful when the first returned product or order equals the expected item specified before execution. `evaluate.py` runs ten controlled cases and generates JSON and Markdown results. A negative case succeeds when no product is returned.

Table 15. Top-1 evaluation

ID	Query	Mode	Expected Top-1	Actual Top-1	Success
E01	black running shoes	Text	Nike Black Running Shoes	Nike Black Running Shoes	Yes
E02	running shoes under 110 dollars	Text	Adidas White Running Shoes	Adidas White Running Shoes	Yes
E03	find blue sports shoes	Voice	Blue Sports Shoes	Blue Sports Shoes	Yes

ID	Query	Mode	Expected Top-1	Actual Top-1	Success
E04	find black headphones	Voice	Black Wireless Headphones	Black Wireless Headphones	Yes
E05	black_leather_bag.png	Image	Black Leather Bag	Black Leather Bag	Yes
E06	green_sports_watch.png	Image	Green Sports Watch	Green Sports Watch	Yes
E07	black shoes + sample image	Multimodal	Nike Black Running Shoes	Nike Black Running Shoes	Yes
E08	nike shoes under 100 dollars	Text	No result	No result	Yes
E09	black shoe	Text	Nike Black Running Shoes	Nike Black Running Shoes	Yes
E10	order O001	Order	O001	O001	Yes

Success Rate = Successful Queries / Total Queries
= 10 / 10
= 100.00%

Table 16. Evaluation summary

Measure	Result
Number of queries	10
Successful queries	10
Success rate	100.00%
Incorrect Top-1 examples in this test set	None
Evaluation scope	Controlled local dataset; image queries reuse catalog images; one negative text case and one order case

These results measure controlled retrieval consistency, not general image recognition or production accuracy. The image demonstration returns Brown Travel Bag among the shoe query's Top-5 results, illustrating a visually misleading lower-ranked result even though the Top-1 test succeeds.

9.3 Automated Tests and Order Demonstration

Table 17. Automated test coverage

Test	Verified behavior	Result
Text query	Black running shoes return product 1 first	Pass
Maximum-price filter	Eligible results cost at most 110 dollars	Pass
Voice query	Blue sports shoes return product 4 first	Pass
Image query	Identical bag image returns product 3 first	Pass
Multimodal query	Black-shoe text plus Nike image returns product 1 first	Pass
Order lookup	O001 returns Shipped status	Pass
Latest order	Customer C001 returns O003	Pass

The updated automated suite contains 13 passing tests, including filter-only queries, conflicting brand and price constraints, empty text validation, and web text, voice, image and order routes. The reproducible commands are `python -m unittest discover -s tests -v` and `python evaluate.py from code/`.

9.4 Web Interface Evidence

The Flask interface reuses the same SearchUI and services as the console. It accepts text, transcript or browser microphone input, image upload, combined text and image, and order requests. Uploaded images are processed through temporary files and removed after each request. The screenshots below were captured with a 1440-pixel Chromium viewport after verifying the expected first result in each mode.

Figure 6. Web home and search form

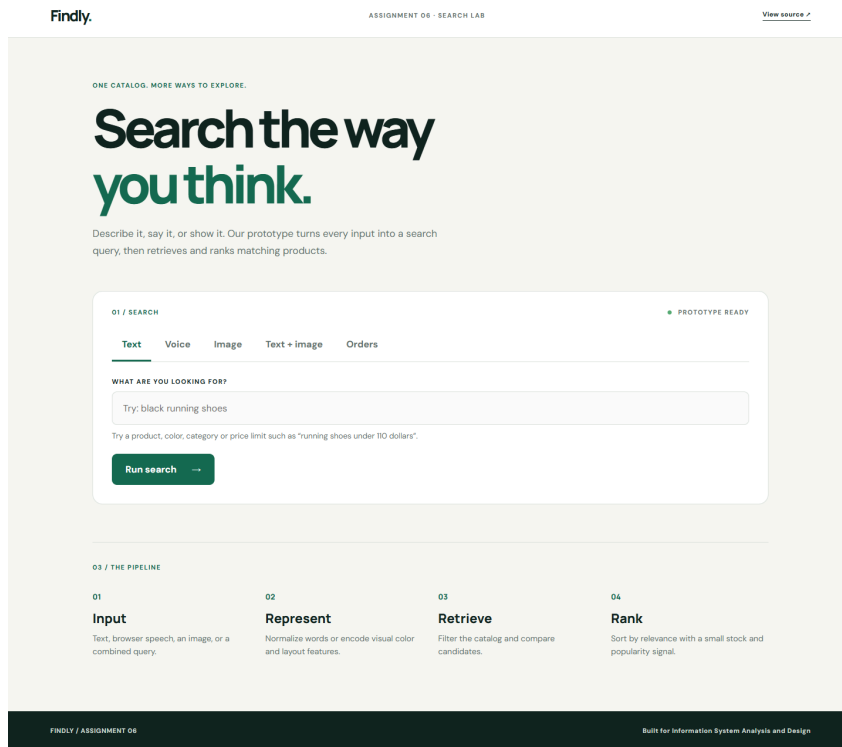


Figure 7. Text search: Nike Black Running Shoes

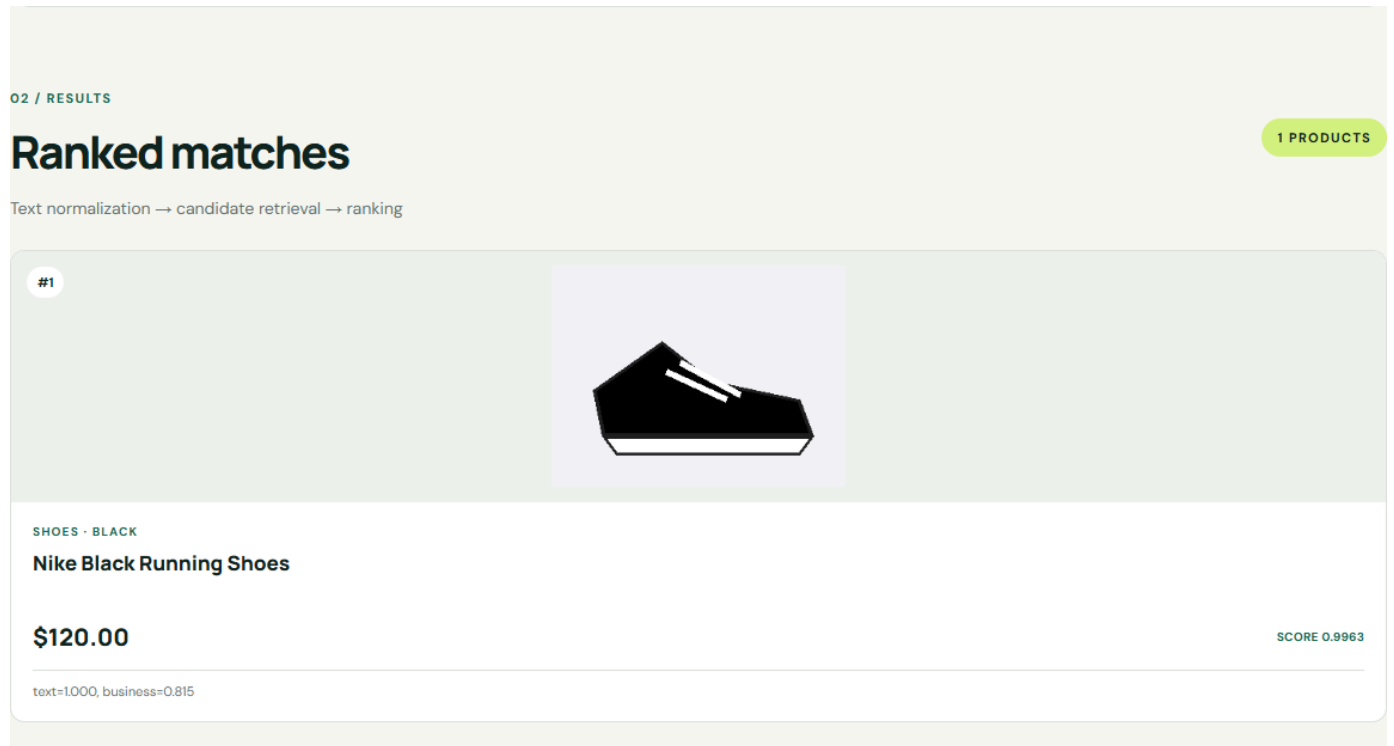


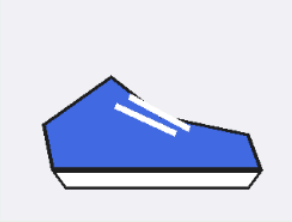
Figure 8. Voice transcript search: Blue Sports Shoes

Ranked matches

1 PRODUCTS

Speech transcription → common query → keyword retrieval → ranking

#1



SHOES · BLUE

Blue Sports Shoes

\$75.00

SCORE 0.9942

text=1.000, business=0.712

Figure 9. Image search: Black Leather Bag

02 / RESULTS

Ranked matches

Image encoding → cosine retrieval → ranking

5 PRODUCTS

#1



BAG · BLACK

Black Leather Bag

\$80.00

SCORE 0.9975

image=1.000, business=0.874

#2



BAG · BROWN

Brown Travel Bag

\$95.00

SCORE 0.9787

image=0.984, business=0.574

#3



SHOES · BLACK

Nike Black Running Shoes

\$120.00

SCORE 0.9736

image=0.977, business=0.815

#4



WATCH · GREEN

Green Sports Watch

\$110.00

SCORE 0.9722

image=0.978, business=0.707

#5



WATCH · BLACK

Black Smart Watch

\$200.00

SCORE 0.9653

image=0.969, business=0.772

Figure 10. Combined text and image search

Ranked matches

1 PRODUCTS

Text + image encoding → score fusion → ranking

#1



SHOES · BLACK

Nike Black Running Shoes

\$120.00

SCORE 0.9963

text=1.000, image=1.000, business=0.815

Figure 11. Order search

Order lookup

1 ORDER

Order request → repository lookup

ORDER 0001

Shipped

Customer C001 · 2026-09-28

\$150.00

10. Discussion

10.1 Compliance with the Assignment

Table 18. Task-to-deliverable mapping

Assignment task	Report evidence	Implementation/artifact
Task 1: Requirements Analysis	Section 3: actor, FR, NFR, inputs, outputs	Requirements tables
Task 2: Use Case Diagram	Section 4 and Figure 1	Visual Paradigm project supplied separately
Task 3: Three-Layer Architecture	Section 5 and Figure 2	presentation/, application/, data_access/
Task 4: Voice Sequence Diagram	Section 6 and Figure 3	Visual Paradigm project supplied separately
Task 5: Python Implementation	Section 7	Repository, text/voice/image retrieval, ranking
Task 6: Demonstration	Section 9.1	main.py demo
Experimental Evaluation	Sections 9.2 and 9.3	evaluate.py, results/, tests/

10.2 Strengths and Limitations

Table 19. Discussion of design choices

Aspect	Strength	Limitation
Architecture	Responsibilities and retrieval/ranking stages are separated	A local prototype rather than a deployed service
Voice	Reuses text processing; browser microphone can fill the transcript	Browser speech support varies and is not measured
Text	Transparent scoring and basic filters	Substring matching; limited language understanding

Aspect	Strength	Limitation
Image	Actual image files and repeatable vectors	Color and layout features can confuse unrelated products
Fusion	Score contributions can be inspected	Equal weights are chosen manually
Evaluation	Reproducible expected Top-1 results	Small test set and repeated catalog images
Orders	ID and latest-order lookup are demonstrated	No authentication or customer-access enforcement

The optional web interface is implemented in Flask. Optional environment settings enable pretrained MobileNet image embeddings (`SEARCH_IMAGE_ENCODER=mobilenet`), SQLite vector storage (`SEARCH_VECTOR_BACKEND=sqlite`), and latent semantic text embeddings (`SEARCH_TEXT_MODE=semantic`). These modes were smoke tested separately. SQLite stores vectors persistently but performs exact scans, not approximate nearest-neighbor indexing. The latent semantic mode is trained on only ten products and can return weakly related items. Browser speech recognition requires a compatible browser and permission. Price, category, stock-aware ranking, order lookup, and multimodal fusion are also implemented.

11. Conclusion

The prototype implements the three required product-search modes and demonstrates their relationship to a three-layer software design. Text and simulated voice queries use keyword retrieval, image queries use vector similarity, and a shared query envelope supports consistent orchestration. Candidate retrieval and ranking remain separate. Ten product records and ten sample images support the demonstration, while thirteen automated tests and ten evaluation queries provide reproducible evidence.

The report includes the required actor, functional and non-functional requirements, use cases, architecture, voice-search sequence, implementation explanation, multimodal method, experimental results, and discussion. Optional fusion, filtering, business scoring, order search, browser interface, and configurable advanced retrieval modes extend the basic prototype. The Visual Paradigm project is supplied separately by the student.